

Anordnung der Pixel eines Bildes und Zugriff auf deren Farbkanäle in Java

Prof. Dr. Klaus Jung



Anordnung der Pixel

- Scanline Anordnung

- Beispiel: Width = 5, Height = 3

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14

```
for(int y = 0; y < height; y++) {
    for(int x = 0; x < width; x++) {
        int pos = y * width + x;
        pixels[pos] = 0xff000000; // black
    }
}
```

Ein Array `int[]` das so lang ist wie die Anzahl der Pixel im Bild.

2 © Klaus Jung

Farbkanäle

- 4 Kanäle ARGB pro Pixel

- A = alpha
- R = rot
- G = grün
- B = blau

- Typischer Wertebereich jedes Kanals

- Ganze Zahlen im Intervall $[0, 255]$
 - 0, 1, 2, 3, ..., 255

- ❑ Ablage jedes Kanals in 1 Byte möglich

- 1 Byte = 8 Bit:

b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀
----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------

3 © Klaus Jung

Rechnen mit Bits und Byte

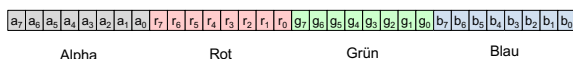
Dezimal	Binär	Hexadezimal
0	0 0 0 0 0 0 0 0	0x00
1	0 0 0 0 0 0 0 1	0x01
2	0 0 0 0 0 0 1 0	0x02
3	0 0 0 0 0 0 1 1	0x03
15	0 0 0 0 1 1 1 1	0x0F
16	0 0 0 1 0 0 0 0	0x10
17	0 0 0 1 0 0 0 1	0x11
32	0 0 1 0 0 0 0 0	0x20
240	1 1 1 1 0 0 0 0	0xF0
255	1 1 1 1 1 1 1 1	0xFF

b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
-------	-------	-------	-------	-------	-------	-------	-------

4 © Klaus Jung

4 Kanäle à 1 Byte = 4 Byte = 32 Bit

- ❑ Ablage ARGB im 32-Bit Integer

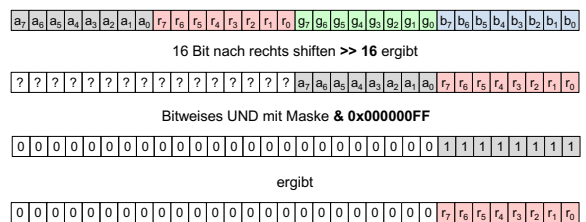


```
int argb = pixels[0];
```

5 © Klaus Jung

Extraktion des Rot-Werts

- ❑ Ablage ARGB im 32-Bit Integer



```
int argb = pixels[0];  
int mask = 0xFF;           // Maske für unterste 8 Bits  
int r = (argb >> 16) & mask; // Rotwert zwischen 0 und 255
```

6 © Klaus Junc

ARGB trennen und zusammensetzen

7

ARGB in einem `int`

```
int argb = pixels[0];

int alpha = (argb >> 24) & 0xff;
int red = (argb >> 16) & 0xff;
int green = (argb >> 8) & 0xff;
int blue = argb & 0xff;

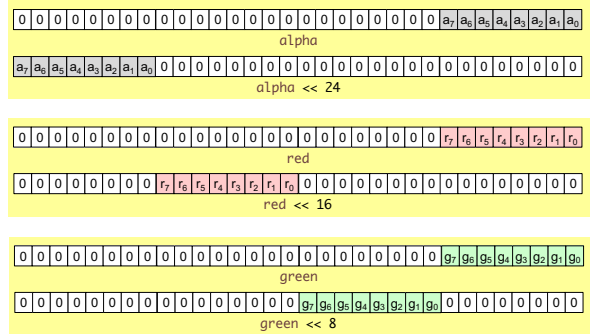
// change color components
// ...

pixels[0] = (alpha << 24) | (red << 16) | (green << 8) | blue;
```

7 © Klaus Jung

Zusammensetzen von ARGB

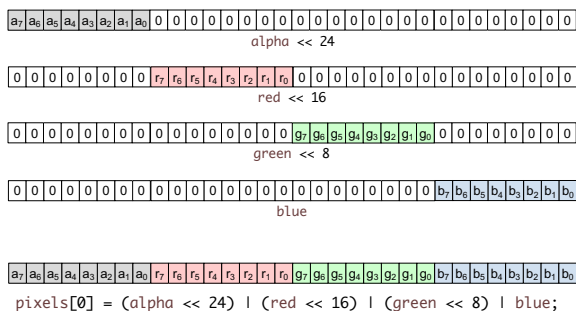
8



8 © Klaus Jung

Zusammensetzen von ARGB

9



```
pixels[0] = (alpha << 24) | (red << 16) | (green << 8) | blue;
```

Was passiert, wenn r, g oder b größer als 255 sind?

9 © Klaus Jung

„Überlaufen“ der Farbwerte verhindern

10

- Falls r, g, b > 255 oder < 0, werden Bits im falschen Kanal gesetzt → **Farbfehler!**
- Gegenmaßnahme:

- Wertebereich begrenzen (Clamping)

```
int red = // ... Rotwert wurde neu berechnet
```

```
if (red > 255) {
    red = 255;
} else if (red < 0) {
    red = 0;
}
```

```
// gleiches für green und blue
```

```
int argb = 0xFF000000 | (red << 16) | (green << 8) | blue;
```

10 © Klaus Jung